

Testing Generator Evaluation & Full Pipeline Testing with the RAG Triad

1. Session Agenda & Strategic Overview

- **Master Plan & Evaluation Suite Architecture:** We are building a comprehensive **RAG Evaluation Suite** across three distinct structural levels:
 1. **Component Level Evaluation:** Testing individual components in strict isolation.
 2. **Pipeline Level Evaluation:** Testing the end-to-end connected workflow.
 3. **Application Level Evaluation:** Evaluating holistic system behavior, custom metrics, and user experience.
 - **Recap of Prior Component Work:** In our previous session, we completed component level evaluation for the **Retriever**. We implemented and tracked two fundamental retrieval metrics: **Recall** and **Precision**.
 - **Today's Strategic Plan:** Our goal today is twofold:
 1. Learn how to build and evaluate the **Generator** component in complete isolation.
 2. Connect the retriever and generator into a complete **RAG Pipeline** and evaluate the pipeline as a single integrated system.
 - **Software Engineering Paradigm:** We follow a strict component by component software development methodology. We do not build the entire system at once and test it at the end. Instead, we build a component, evaluate it in isolation, refine it, build the next component, evaluate it in isolation, connect them into a pipeline and evaluate the full pipeline.
-

2. Building the Generator Component in Isolation

2.1 Theoretical Foundations of a Generator

- **Definition & Core Function:** In a RAG architecture, the **Generator** is a functional component whose sole responsibility is to produce a natural language answer based strictly on user input and retrieved documents.
- **Input Output Contract:** The generator operates as a function accepting exactly two inputs and producing one output:
 - **Input 1:** User Question / Query.
 - **Input 2:** Relevant Context Documents (fetched by the retriever).
 - **Output:** Generated Answer.

2.2 Implementation Details (`generator.py`)

- **LLM Selection:** We select **GPT-4o-mini** as our baseline language model because it is cost effective.
- **Temperature Parameter:** We explicitly set `temperature = 0`. Setting the temperature to zero ensures deterministic, consistent outputs, which is essential during evaluation runs.
- **System Prompt Design & Rules:** We define a tight system prompt to constrain the generator's behavior:
 - **Persona:** A helpful teaching assistant for a course on LLM evaluations.
 - **Strict Constraint 1:** Answer the student's question using **only** the provided context, do not add outside knowledge.
 - **Strict Constraint 2 (Hallucination Guardrail):** If the provided context lacks sufficient information to answer the question, explicitly output: *"I don't have enough information in the course material to answer that."*
 - **Formatting Instruction:** Keep the answer clear and concise.
- **Chain Construction:** Using LangChain, we construct a standard chain consisting of the prompt template, the LLM, and `StrOutputParser` to extract string outputs.
- **Functional Interface:** We encapsulate the chain inside a `generate(query, context)` function that takes the raw query and context strings and returns the generated answer.

2.3 Verification & Dry Run

- **Dry Run Setup:** To verify that our generator code executes without runtime errors, we test `generator.py` using a manually constructed dummy context and query.
 - **Test Query:** *"What is online eval?"*.
 - **Dummy Context:** Text defining online evaluation as testing on live production traffic without ground truth answer keys.
 - **Execution Verification:** The generator correctly synthesized the definition directly from the provided dummy context, confirming that our component code works cleanly.
-

3. Evaluating the Generator: First Principles Failure Modes & Key Metrics

3.1 First Principles Evaluation Framework

- **Failure Mode Analysis:** To evaluate any system from first principles, we must identify all the specific ways that system can fail.
- **Primary Failure Modes of a Generator:** A generator component primarily fails in two major

ways:

1. Generating an **Unfaithful Response** (hallucinating outside information).
2. Generating an **Irrelevant Response** (failing to directly answer the query despite using context).

3.2 Failure Mode 1: Unfaithful Responses & Faithfulness

- **Definition:** An unfaithful response occurs when the generator reads the retrieved context but invents or injects additional information from its own pre-trained internal knowledge that is not present in the context.
- **Production Risks:** Unfaithful responses are extremely dangerous in commercial applications. For example, in our Air Canada case study, the customer support chatbot invented an unauthorized refund policy because the generator fabricated information beyond its provided context.
- **Metric 1: Faithfulness:** **Faithfulness** measures the degree to which the generated answer is strictly derived from and supported by the provided context.
- **Faithfulness vs. Correctness:**
 - **Faithfulness** strictly measures adherence to the context. If the retrieved context contains incorrect facts and the generator outputs those exact incorrect facts without adding anything extra, the generator is **100% faithful**.
 - Faithfulness does not measure whether the answer is factually correct in the real world, it only promises that the generator did not invent information beyond the provided context.

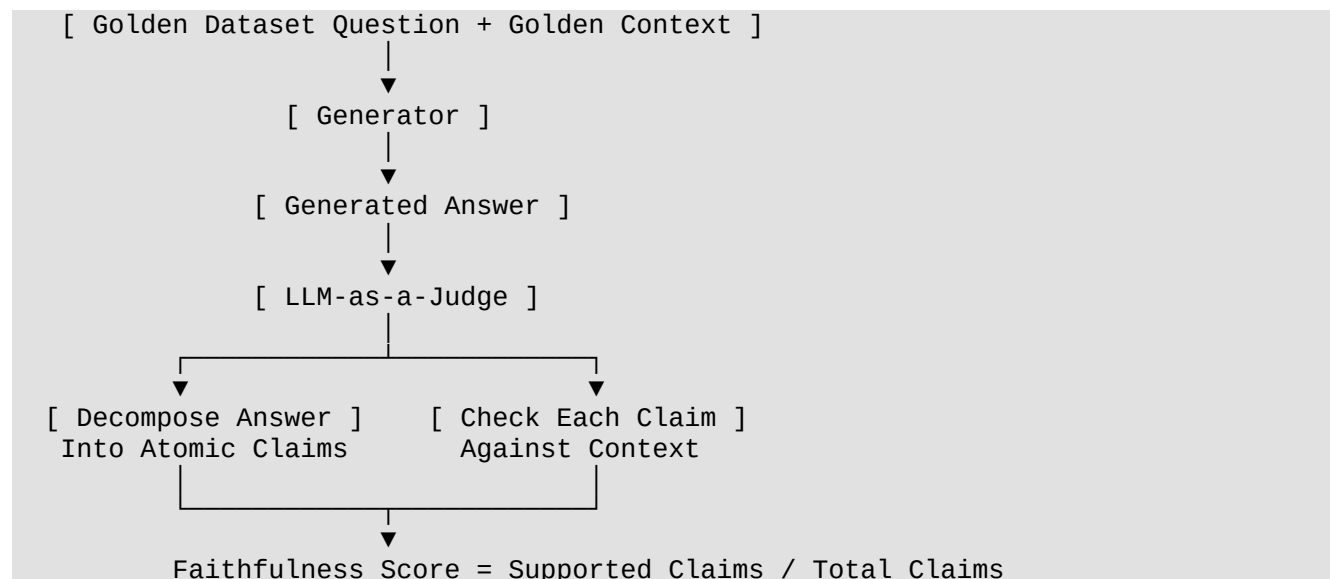
3.3 Failure Mode 2: Irrelevant Responses & Answer Relevance

- **Definition:** An irrelevant response occurs when the generator produces an answer derived entirely from the context (it is 100% faithful), but the answer fails to actually address the user's question.
- **Illustrative Scenario:**
 - **User Query:** *"Does the CampusX AI Engineering program include live classes?"*.
 - **Retrieved Context:** *"The AI Engineering program includes recorded lessons, coding assignments, projects, and weekly doubt solving sessions."* (Note: The context does not mention live classes).
 - **Unhelpful/Irrelevant Output:** *"The program includes coding assignments, projects, recorded lessons, and weekly doubt solving sessions."* (Faithful to context, but fails to answer whether live classes exist).
 - **Helpful/Relevant Output:** *"The provided context does not confirm that the program includes live classes; it only mentions recorded lessons, coding assignments, projects, and weekly doubt solving sessions."*

- **Metric 2: Answer Relevance:** **Answer Relevance** measures how directly and effectively the generated answer addresses the user's input query.
-

4. Mathematics & Step-by-Step Calculation of Generator Metrics

4.1 Calculating Faithfulness



- **Dataset Requirement:** Evaluating faithfulness requires a dedicated **Golden Dataset** containing two fields:
 1. **Question:** The user query.
 2. **Golden Context:** Ideal, manually verified context chunks fetched across the database.
- **Isolated Testing Setup:** To evaluate the generator in isolation, we bypass the retriever completely and pass the **Golden Context** directly from our dataset into the generator.
- **Step-by-Step Calculation Process:**
 1. We feed the dataset Question and Golden Context to the generator to produce a generated answer.
 2. An **LLM-as-a-Judge** decomposes the generated answer into individual atomic claims.
 3. The LLM Judge verifies each claim against the Golden Context to determine if the claim is explicitly supported by the context.
- **Mathematical Formula:**
$$\text{Faithfulness Score} = \frac{\text{Number of Claims Supported by Context}}{\text{Total Number of Claims in Generated Answer}}$$
- **Numerical Example:**
 1. Generated Answer contains 5 atomic claims.
 2. Upon inspection against Golden Context, Claims 1, 2, and 4 are found in the context,

while Claims 3 and 5 are missing.

3. $\text{Faithfulness Score} = \frac{3}{5} = 0.60$ (or 60%)\$.

- **Dataset Averaging:** We compute the Faithfulness score for every individual question in our golden dataset and calculate the mean average score across the dataset.
- **Faithfulness vs. Retriever Recall:** Faithfulness is conceptually the inverse of Recall. Recall evaluates fetched context against ground truth information, whereas Faithfulness evaluates generated answers against provided context.

4.2 Calculating Answer Relevance

- **Reference Free Property:** Answer Relevance is a **reference free evaluation** metric because it does not require ground truth reference answers or reference context chunks to evaluate.
- **Step-by-Step Calculation Process:**
 1. We pass a query to the generator to get an answer.
 2. An **LLM-as-a-Judge** breaks down the generated answer into individual atomic claims.
 3. For each claim, the LLM Judge checks whether the claim directly helps answer the input query or if it is off topic filler.
- $\text{Answer Relevance Score} = \frac{\text{Number of Relevant Claims}}{\text{Total Number of Claims in Generated Answer}}$
- **Numerical Example:**
 1. **Query:** "What does it mean for a benchmark to get saturated?"
 2. **Generated Answer:** "A benchmark gets saturated when model scores cluster high and close together, so it can no longer tell models apart. Benchmark contamination happens when test data leaks into training data."
 3. **Claim Breakdown:**
 - Claim 1: Benchmark saturation occurs when scores cluster high together. (Relevant).
 - Claim 2: The benchmark can no longer differentiate between models. (Relevant).
 - Claim 3: Benchmark contamination happens when test data leaks into training. (Irrelevant to query).
 4. $\text{Answer Relevance Score} = \frac{2}{3} \approx 0.67$ (or 67%)\$

4.3 Technical Nuances & Student QA

- **Handling Missed Claims:** If the generator omits key facts required to answer the query, the generated claims will score lower in relevance because incomplete/off topic responses contain a higher ratio of non helpful claims.
- **Single vs. Separate LLM Judges:** In DeepEval implementations, the exact same LLM model

is used for claim decomposition and claim analysis across independent API calls.

- **LLM Judge Errors & Relative Consistency:**
 - LLM judges are subject to false positives and false negatives.
 - However, because the same judge model is used consistently across evaluation runs, systematic judge bias remains constant. This allows reliable relative comparison between different prompts, settings, or models.
 - Mitigation: Use powerful, high reasoning judge models (e.g., GPT-4o-mini or GPT-4o).
-

5. Curating the Golden Dataset for Generator Evaluation

- **Dataset Construction Methodology:**
 1. **Exporting Vector Store:** We ran a script (`export_chroma_chunks.py`) to dump all 862 vector store chunks from ChromaDB into a JSON file.
 2. **Step-by-Step LLM Prompting:** We uploaded the 862 chunks to Claude and instructed it to generate dataset records one at a time (one question and its corresponding golden context).
 3. **Human in the Loop Verification:** I personally reviewed every question and its extracted golden context to verify factual accuracy against my actual course lectures.
 - **Dataset Artifact:** The verified records were saved into `faithfulness_dataset.json` containing 15 curated question-context pairs.
 - **Alternative Creation Methods:**
 1. Purely manual creation by subject matter experts.
 2. LLM assisted generation with human in the loop review.
 3. Automated synthetic data generators (such as DeepEval Synthesizer).
-

6. Implementation & Initial Evaluation of Generator using DeepEval

6.1 Evaluation Code Implementation (`eval_generator.py`)

- **Core Logic:**
 1. Load `faithfulness_dataset.json`.
 2. Initialize the evaluation judge model using `GPT-4o-mini` and set pass/fail threshold scores.
 3. Loop over dataset entries, passing the question and golden context to `generate()`,

and wrap the outputs into an `LLMTestCase` object:

- `input`: Question from golden dataset.
 - `actual_output`: Generated answer string from `generate()`.
 - `retrieval_context`: Golden Context from golden dataset.
4. Instantiate DeepEval's built in `FaithfulnessMetric` and `AnswerRelevancyMetric` with `include_reason=True`.
 5. Call DeepEval's `evaluate()` function to execute tests in parallel across test cases.

6.2 Initial Evaluation Results

- **Initial Scores:**
 - **Faithfulness Score: 91% (0.91).**
 - **Answer Relevance Score: 73% (0.73).**
 - **Diagnostic Analysis:**
 - **Why Faithfulness was High Initially (91%):** Modern LLMs have strong instruction following capabilities. When provided with explicit context and instructed not to hallucinate, the LLM naturally restricts itself to the provided text.
 - **Why Answer Relevance was Low Initially (73%):** The baseline system prompt was too simple. The generator frequently echoed context fragments or introduced slight tangents instead of directly addressing the student's core question.
-

7. Generator Optimization via System Prompt Iteration

7.1 Levers for Generator Improvement

To improve generator performance on Faithfulness and Answer Relevance, we have two primary options:

1. **Model Upgrading:** Switch to a larger, more capable base model with stronger instruction following abilities.
2. **System Prompt Engineering:** Refine rules, guidelines, and constraints in the system prompt.

7.2 System Prompt Iteration Process

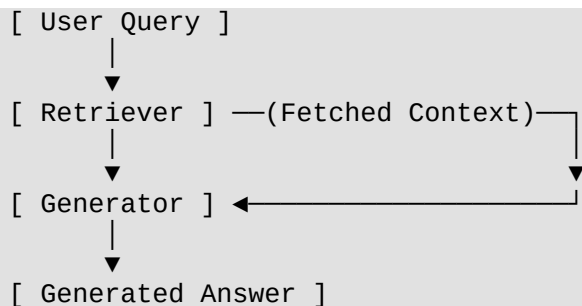
- **Error Analysis Workflow:** I examined every failed test case from the initial DeepEval run, extracted the error reasons provided by the judge, fed them into Claude and refined the system prompt iteratively across 3 4 testing cycles.
- **Refined System Prompt Rules:**
 - Use *only* information present in the context; do not add outside knowledge.

- Do not strengthen or overstate claims (e.g., if context states two things are different, do not claim they are separate formal methods).
- Recognize that context originates from an informal lecture transcript; synthesize and rephrase information without requiring exact matching wording from the question.

7.3 Post Optimization Evaluation Results

- **Re evaluation Benchmarks:**
 - **Faithfulness Score:** Increased from 91% to **96% (0.96)**.
 - **Answer Relevance Score:** Increased significantly from 73% to **92% (0.92)**.
- **Overfitting Warning:** System prompts must remain general. Over customizing prompt instructions to pass specific failing items in a small test set risks overfitting, causing performance to degrade when exposed to live, noisy retriever contexts.
- **Experiment Tracking via Confident AI:** Evaluation runs, system prompts, hyperparameter configs, and detailed failure logs can be synced directly to Confident AI (DeepEval's platform) to maintain experiment tracking.

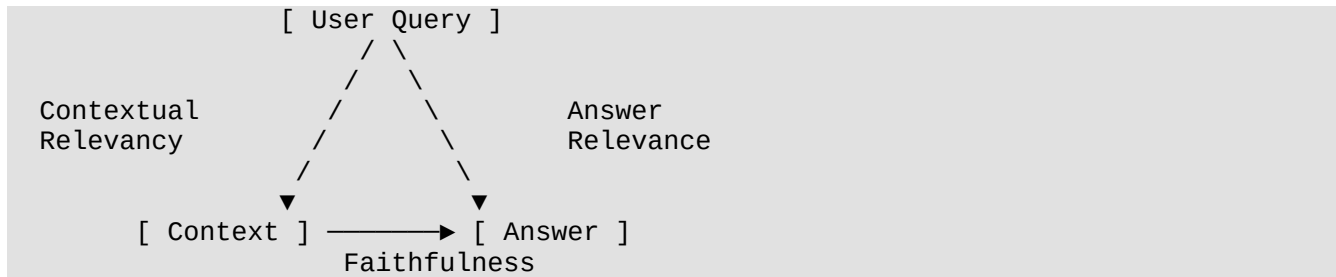
8. Connecting Components & Evaluating the Full RAG Pipeline



8.1 Building the End-to-End Pipeline (rag_pipeline.py)

- **Pipeline Integration:** We connect our Re-ranking Retriever and our optimized Generator into a single unified `RAGPipeline` class.
- **Execution Flow:**
 1. The user query is passed to the re-ranking retriever to fetch context chunks.
 2. Retrieved chunks are concatenated into a single context string.
 3. The user query and retrieved context string are passed into `generate()`.
 4. The generator returns the final answer string.
- **Verification Run:** Tested with a live query ("What is drift and why does it matter after deployment?"), successfully retrieving 5 real chunks and returning a synthesized response.

8.2 The RAG Triad Framework



- **Framework Overview:** Evaluates the 3 structural vertices of a RAG pipeline: **Query**, **Context**, and **Answer**.
- **The 3 Triad Relationships:**
 1. **Faithfulness (Answer ↔ Context):** Measures if the generated answer is grounded entirely in retrieved context.
 2. **Answer Relevance (Answer ↔ Query):** Measures if the generated answer addresses the user query.
 3. **Contextual Relevancy (Context ↔ Query):** Measures if the retrieved context is relevant to the user query.

8.3 Contextual Relevancy Calculation

- **Definition:** Measures what proportion of sentence level claims within retrieved context chunks are directly relevant to answering the query.
- **Step-by-Step Calculation:**
 1. Query is sent to retriever, fetching (K) context chunks (e.g., (K=5)).
 2. An LLM Judge decomposes all retrieved context chunks into atomic context claims.
 3. The LLM Judge inspects each context claim individually and determines whether it helps answer the query.
 4.
$$\text{Contextual Relevancy Score} = \frac{\text{Number of Relevant Context Claims}}{\text{Total Number of Context Claims Across Fetched Chunks}}$$

9. Pipeline Evaluation & The "Curious Case of the Retriever" Paradox

9.1 Pipeline Evaluation Setup (eval_rag_pipeline.py)

- **Component vs. Pipeline Evaluation Setup:** In component testing, the generator received static Golden Context. In pipeline testing, `retrieved_context` is populated dynamically by the active retriever component.

- **Metrics Suite:** We execute `FaithfulnessMetric`, `AnswerRelevancyMetric`, and `ContextualRelevancyMetric` concurrently across the dataset.

9.2 Pipeline Evaluation Results

- **Aggregate Pipeline Benchmarks:**
 - **Faithfulness Score: 93% (0.93).**
 - **Answer Relevance Score: 86% (0.86).**
 - **Contextual Relevancy Score: 42% (0.42).**

9.3 The Retriever Paradox

- **The Contradiction:**
 - Standalone Retriever Metrics: **Recall = 99%, Precision = 89%.**
 - Pipeline Contextual Relevancy: **42%.**
- **Analyzing the Paradox:** How can a retriever exhibit 99% Recall and 89% Precision in component testing, but register only 42% Contextual Relevancy in full pipeline testing?.

9.4 Paradox Resolution: Macro Level Chunk Utility vs. Micro Level Intra Chunk Noise

RETRIEVED CHUNK (Precision = PASS / Useful at Chunk Level)		
Sentence 1: Useful fact answering query (RELEVANT)		
Sentence 2: Informal lecture tangent (NOISE)		
Sentence 3: Conversational transition (NOISE)		
Sentence 4: Administrative course update (NOISE)		
Result: Chunk Precision = 1/1 (100%), Context Relevancy = 1/4 (25%)		

- **Precision Mechanics:** **Precision** operates at the **macro chunk level**. If a retrieved chunk contains 5 sentences, and even 1 sentence helps answer the query, the entire chunk is marked as useful, contributing positively to Precision.
- **Contextual Relevancy Mechanics:** **Contextual Relevancy** operates at the **micro sentence/claim level**. The LLM Judge breaks the chunk into individual sentences/claims. If 1 sentence is useful but the remaining 4 sentences contain lecture banter, transitions, or off topic explanation, Contextual Relevancy scores that chunk as $\frac{1}{5} = 20\%$.
- **Root Cause:** The low Contextual Relevancy score (42%) reveals that our retrieved chunks contain significant intra chunk noise. While the retriever successfully selects correct chunks

(yielding high Recall and Precision), those chunks carry excess irrelevant text into the prompt context window.

- **Remediation Strategy:** Reduce **Chunk Size** during vector ingestion. Smaller chunk sizes reduce intra chunk noise and boost Contextual Relevancy, though chunk size must be balanced to avoid dropping necessary surrounding context.
-

10. Summary of Progress & Roadmap

10.1 Completed Evaluation Levels

- **Component Level Evaluation (Complete):**
 - **Retriever:** Evaluated using **Recall** (99%) and **Precision** (89%).
 - **Generator:** Evaluated using **Faithfulness** (96%) and **Answer Relevance** (92%).
- **Pipeline Level Evaluation (Complete):**
 - **RAG Triad:** Evaluated using **Faithfulness** (93%), **Answer Relevance** (86%), and **Contextual Relevancy** (42%).

10.2 Future Roadmap (Application Level Evaluation)

- **Application Level Metrics:**
 - **Correctness:** Evaluating real world factual correctness against true ground truth answers.
 - **Completeness:** Checking if all requested sub aspects of a query were addressed.
 - **Style Matching:** Evaluating persona and tone alignment against specific instructor or brand standards.
 - **Safety & Ops Evals:** Guardrails, toxicity, latency, cost tracking and security evaluations.
 - **G-Eval & Online Evals:** Custom metric definitions using G-Eval frameworks, regression testing and production monitoring on live traffic.